Vidyavardhini's

College of Engineering & Technology

Vasai Road (W)

Department of Computer Engineering

Laboratory Journal

Semester	VIII	Class	B.E.
Course Code	**CSL801**	Academic Year	2025-26
Course Name	Distributed Computing		
Name of Faculty	Dr. Swati Varma & Ms. Vinal Waghela		

Vidyavardhini's College of Engineering & Technology

Vision

To be a premier institution of technical education, aiming at becoming a valuable resource for industry and society.

Mission

- To provide technologically inspiring environment for learning.
- To promote creativity, innovation and professional activities.
- To inculcate ethical and moral values.
- To cater personal, professional and societal needs through quality education.

Department Vision:

To evolve as a center of excellence in the field of Computer Engineering to cater to industrial and societal needs.

Department Mission:

- To provide quality technical education with the aid of modern resources.
- Inculcate creative thinking through innovative ideas and project development.
- To encourage life-long learning, leadership skills, entrepreneurship skills with ethical & moral values.

Program Education Objectives (PEOs):

PEO1: To facilitate learners with a sound foundation in the mathematical, scientific and engineering fundamentals to accomplish professional excellence and succeed in higher studies in Computer Engineering domain

PEO2: To enable learners to use modern tools effectively to solve real-life problems in the field of Computer Engineering.

PEO3: To equip learners with extensive education necessary to understand the impact of computer technology in a global and social context.

PEO4: To inculcate professional and ethical attitude, leadership qualities, commitment to societal responsibilities and prepare the learners for life-long learning to build up a successful career in Computer Engineering.

Program Specific Outcomes (PSOs):

PSO1: Analyze problems and design applications of database, networking, security, web technology, cloud computing, machine learning using mathematical skills, and computational tools.

PSO2: Develop computer-based systems to provide solutions for organizational, societal problems by working in multidisciplinary teams and pursue a career in the IT industry.

Program Outcomes (POs):

Engineering Graduates will be able to:

- **PO1. Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

- **PO2. Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

- **PO3. Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

- **PO4. Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

- **PO5. Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

- **PO6. The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

- **PO7. Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

- **PO8. Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

- **PO9. Individual and teamwork:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

- **PO10. Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

- **PO11. Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

- **PO12. Life-long learning:** Recognize the need for and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Course Objectives

1	To understand basic underlying concepts of forming distributed systems
2	To learn the concept of clock Synchronization
3	To learn Election Algorithm
4	To explore mutual exclusion algorithms and deadlock handling in the distributed system
5	To study resource allocation and management
6	To understand the Distributed File System

Course Outcomes

CO	At the end of the course student will be able to	Action verb	Bloom Level
CSL801.1	Implement message-oriented Communication or RPC/RMI based client-server programs	Implement	APPLY (Level 3)
CSL801.2	Implement techniques for clock synchronization	Implement	APPLY (Level 3)
CSL801.3	Implement techniques for election algorithms	Implement	APPLY (Level 3)
CSL801.4	Implement mutual exclusion and deadlock handling techniques	Implement	APPLY (Level 3)
CSL801.5	Implement techniques of resource and process management	Implement	APPLY (Level 3)
CSL801.6	Implement distributed file systems	Implement	APPLY (Level 3)

Exp. No.	Name of Experiment
1	Implement inter-process communication using Sockets
2	Implement Client/Server using RPC/RMI
3	Implement Group Communication
4	Implement Vector Clock Synchronization algorithms*
5	Implement Election Algorithm
6	Implement Mutual Exclusion Algorithm
7	Implement Deadlock Management in Distributed System
8	Implement Task Assignment Approach
9	Implement Distributed shared Memory
10	Demonstrate Distributed File System

Mapping of Experiments with Course Outcomes

Experiments	Course Outcomes					
	CSL801.1	CSL80.2	CSL801.3	CSL801.4	CSL801.5	CSL801.6
Implement inter-process communication using Sockets	3	-	-	-	-	-
Implement Client/Server using RPC/RMI	3	-	-	-	-	-
Implement Group Communication	3	-	-	-	-	-
Implement Vector Clock Synchronization algorithms*	-	3	-	-	-	-
Implement Election Algorithm	-	-	3	-	-	-
Implement Mutual Exclusion Algorithm	-	-	-	3	-	-

Implement Deadlock Management in Distributed System	-	-	-	3	-	-
Implement Task Assignment Approach	-	-	-	-	3	-
Implement Distributed shared Memory	-	-	-	-	-	3
Demonstrate Distributed File System	-	-	-	-	-	3

Enter correlation level 1, 2 or 3 as defined below
1: Slight (Low) 2: Moderate (Medium) 3: Substantial (High)
If there is no correlation put "—".

Laboratory Name :

Lab Outcomes:

CSL801.1	Implement message-oriented Communication or RPC/RMI based client-server programs.	APPLY
CSL801.2	Implement techniques for clock synchronization	APPLY
CSL801.3	Implement techniques for election algorithms	APPLY
CSL801.4	Implement mutual exclusion and deadlock handling techniques	APPLY
CSL801.5	Implement techniques of resource and process management	APPLY
CSL801.6	Implement distributed file systems	APPLY

List of Laboratory Experiment

Exp. No.	Name of Experiment
1	Implement inter-process communication using Sockets.
2	Implement Client/Server using RPC/RMI
3*	Implement Group Communication
4 *	Implement Vector Clock Synchronization algorithms
5	Implement Election Algorithm
6	Implement Mutual Exclusion Algorithm
7	Implement Deadlock Management in Distributed System
8	Implement Task Assignment Approach
9	Implement Distributed shared Memory
10	Demonstrate Distributed File System

Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

Academic Year: 2025-26

Class:	BE	**Semester:**	VIII
Course Code:	CSL801	**Course Name:**	Distributed Computing Lab

Name of Student:	Tanishka Jayesh Das
Roll No.:	34
Division:	1
Experiment No.:	1
Title of Experiment:	Implement inter-process communication using Sockets
Date of Submission:	6/1/2026
Date of Correction:	14/1/2026

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	4-5	2-3	1
Understanding	4-5	2-3	1
Journal work and timely submission	8-10	5-8	1-4

Checked by

Name of Faculty : **Dr. Swati Varma**

Signature :

Date :

EXPERIMENT 1

Aim: To implement inter-process communication

Objective: Develop a program to implement inter-process communication using socket programming

Theory:-

URLs and URLConnections provide a relatively high-level mechanism for accessing resources on the Internet. Sometimes your programs require lower-level network communication, for example, when you want to write a client-server application.

In client-server applications, the server provides some service, such as processing database queries or sending out current stock prices. The client uses the service provided by the server, either displaying database query results to the user or making stock purchase recommendations to an investor. The communication that occurs between the client and the server must be reliable. That is, no data can be dropped and it must arrive on the client side in the same order in which the server sent it.

TCP provides a reliable, point-to-point communication channel that client-server applications on the Internet use to communicate with each other. To communicate over TCP, a client program and a server program establish a connection to one another. Each program binds a socket to its end of the connection. To communicate, the client and the server each reads from and writes to the socket bound to the connection.

What Is a Socket?

A socket is one end-point of a two-way communication link between two programs running on the network. Socket classes are used to represent the connection between a client program and a server program. The java.net package provides two classes--Socket and ServerSocket--that implement the client side of the connection and the server side of the connection, respectively.

Socket class

A socket is simply an endpoint for communications between the machines. The Socket class can be used to create a socket.

Important methods

Method	Description
1) public InputStream getInputStream()	returns the InputStream attached with this socket.
2) public OutputStream getOutputStream()	returns the OutputStream attached with this socket.
3) public synchronized void close()	closes this socket

ServerSocket class

The ServerSocket class can be used to create a server socket. This object is used to establish communication with the clients.

Important methods

Method	Description
1) public Socket accept()	returns the socket and establish a connection between server and client.
2) public synchronized void close()	closes the server socket.

Code and output:

Code:

```
import socket
import threading
import time

  ---------------                    ---------------
#             SERVER FUNCTION
def server_program():
    server_socket          =          socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
    host = "127.0.0.1"
    port = 5000

    server_socket.bind((host, port))
    server_socket.listen(1)
    print("Server: Waiting for connection...")

    conn, addr = server_socket.accept()
    print("Server: Connected to", addr)

    message = conn.recv(1024).decode()
    print("Server: Received ->", message)

    response = "Hello Client, your message is received!"
    conn.send(response.encode())

  ---------------                    ---------------
    conn.close()
    server_socket.close()

#             CLIENT FUNCTION
def client_program():
    time.sleep(1)  # Ensures server starts first

    client_socket          =          socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
    host = "127.0.0.1"
    port = 5000
    client_socket.connect((host, port))
```

```
    response = client_socket.recv(1024).decode()
    print("Client: Received ->", response)

    client_socket.close()

# --------------- MAIN ---------------
server_thread = threading.Thread(target=server_program)
client_thread = threading.Thread(target=client_program)

server_thread.start()
client_thread.start()

server_thread.join()
client_thread.join()
```

OUTPUT:

```
Server: Waiting for connection...
Server: Connected to ('127.0.0.1', 57330)
Server: Received -> Hello Server, this is Client!
Client: Received -> Hello Client, your message is received!
```

Conclusion:

In this experiment, inter-process communication was implemented using socket programming. The client and server successfully exchanged messages through a TCP connection. The client sent **"Hello Server, this is Client!"** and the server replied **"Hello Client, your message is received!"**.

Socket programming allows communication between processes on the same or different computers through a network. Unlike other IPC methods like pipes or shared memory, sockets support long-distance and network communication. Hence, socket programming is useful for client–server and distributed applications.

Class:	BE	**Semester:**	VIII
Course Code:	CSL801	**Course Name:**	Distributed Computing Lab

Name of Student:	Tanishka Jayesh Das
Roll No.:	34
Division:	1
Experiment No.:	2
Title of Experiment:	Implement Client/Server using RPC/RMI
Date of Submission:	14/1/2026
Date of Correction:	22/1/2026

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	4-5	2-3	1
Understanding	4-5	2-3	1
Journal work and timely submission	8-10	5-8	1-4

Checked by

Name of Faculty : **Dr. Swati Varma**

Signature :

Date :

EXPERIMENT 2

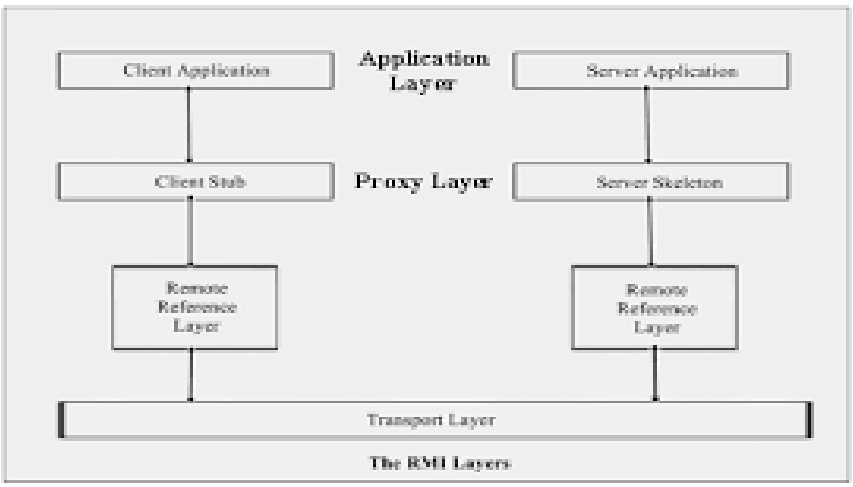
Aim: To implement Client/Server using RPC/RMI

Objective: Develop a program to implement Client/Server using RPC/RMI

Theory:
The **RMI** (Remote Method Invocation) is an API that provides a mechanism to create distributed application in java. It allows an object to invoke methods on an object running in another JVM. It provides remote communication between the applications using two objects *stub* and *skeleton*.

Understanding stub and skeleton:
RMI uses stub and skeleton object for communication with the remote object.
A **remote object** is an object whose method can be invoked from another JVM. Let's understand the stub and skeleton objects:
Stub
The stub is an object, acts as a gateway for the client side. All the outgoing requests are routed through it. It resides at the client side and represents the remote object. When the caller invokes method on the stub object, it does the following tasks:
1. It initiates a connection with remote Virtual Machine (JVM),
2. It writes and transmits (marshals) the parameters to the remote Virtual Machine (JVM),
3. It waits for the result
4. It reads (unmarshals) the return value or exception, and
5. It finally, returns the value to the caller.

Skeleton
The skeleton is an object, acts as a gateway for the server side object. All the incoming requests are routed through it. When the skeleton receives the incoming request, it does the following tasks:
1. It reads the parameter for the remote method
2. It invokes the method on the actual remote object, and
3. It writes and transmits (marshals) the result to the caller.



Steps to write the RMI program

6 steps to write the RMI program.

1. Create the remote interface
2. Provide the implementation of the remote interface
3. Compile the implementation class and create the stub and skeleton objects using the rmic tool
4. Start the registry service by rmiregistry tool
5. Create and start the remote application
6. Create and start the client application

Code and output:

Code:

```
import socket
import threading
import pickle
import time

# ------------------------- RPC SERVER -------------------------
class RPCServer:
    def add(self, a, b):
        return a + b

    def multiply(self, a, b):
        return a * b

def server_program():
server = RPCServer()

    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.bind(("127.0.0.1", 6000))
    s.listen(1)

    print("RPC Server: Waiting for client...")

    conn, addr = s.accept()
    print("RPC Server: Client connected from", addr)

    while True:
        data = conn.recv(1024)
        if not data:
            break
      function name, args = pickle.loads(data)
```

```
        print(f"RPC Server: Request -> {function_name}{args}")

        # Call actual method
        result = getattr(server, function_name)(*args)

        conn.send(pickle.dumps(result))

    conn.close()
    s.close()


  ------------------------                ------------------------
#                    RPC CLIENT
def client_program():
    time.sleep(1)  # wait for server

    c = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    c.connect(("127.0.0.1", 6000))

    # Remote call 1
    request = ("add", (10, 20))
    c.send(pickle.dumps(request))
    result = pickle.loads(c.recv(1024))
    print("RPC Client: add(10,20) =", result)

    # Remote call 2
    request = ("multiply", (5, 6))
    c.send(pickle.dumps(request))
    result = pickle.loads(c.recv(1024))
    print("RPC Client: multiply(5,6) =", result)

    c.close()
  ------------------------        ------------------------


#                    MAIN
server_thread = threading.Thread(target=server_program)
client_thread = threading.Thread(target=client_program)

server_thread.start()
client_thread.start()

server_thread.join()
client_thread.join()
```

Output:

```
RPC Server: Waiting for client...
RPC Server: Client connected from ('127.0.0.1', 45150)
RPC Server: Request -> add(10, 20)
RPC Client: add(10,20) = 30
RPC Server: Request -> multiply(5, 6)
RPC Client: multiply(5,6) = 30
```

Conclusion:

In this experiment, the client was able to access the services provided by the server using the RPC/RMI concept. The server exposed its methods such as **add()** and **multiply()**, and the client invoked these methods remotely as if they were local functions. The client sent the method name and parameters to the server, and the server executed the requested method and returned the result.

This shows that RPC/RMI allows transparent communication between client and server, enabling distributed applications to share services easily without the client knowing the internal working of the server.

Class:	BE	**Semester:**	VIII
Course Code:	CSL801	**Course Name:**	Distributed Computing Lab

Name of Student:	Tanishka Jayesh Das
Roll No.:	34
Division:	1
Experiment No.:	3
Title of Experiment:	Implement Group Communication
Date of Submission:	22/1/2026
Date of Correction:	29/1/2026

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	4-5	2-3	1
Understanding	4-5	2-3	1
Journal work and timely submission	8-10	5-8	1-4

Checked by

Name of Faculty : **Dr. Swati Varma**

Signature :

Date :

EXPERIMENT 3

Aim: To implement Group Communication

Objective: Develop a program to implement Group Communication

Theory:

Group communication refers to the process of exchanging information among a group of processes in a distributed system. In a group communication scenario, a message is sent by a process to a group of processes, and all the processes in the group receive the message.

Group communication is used in a wide range of distributed applications, such as group chat, distributed file systems, and distributed databases.

Multicast is a popular group communication protocol that involves the transmission of a message to a group of processes using a single send operation. The message is delivered to all the processes in the group, which allows for efficient dissemination of information among multiple processes. Multicast can be implemented using various network protocols, such as User Datagram Protocol (UDP) and Transmission Control Protocol (TCP).

The multicast datagram socket class is useful for sending and receiving IP multicast packets. A MulticastSocket is a (UDP) DatagramSocket, with additional capabilities for joining "groups" of other multicast hosts on the internet.
A multicast group is specified by a class D IP address and by a standard UDP port number. Class D IP addresses are in the range 224.0.0.0 to 239.255.255.255, inclusive. The address 224.0.0.0 is reserved and should not be used.

To join a multicast group, first create a MulticastSocket with the desired port, then invoke the joinGroup(InetAddress groupAddr) method.

Code and output:

```
import socket
import struct
import threading

MULTICAST_GROUP = '224.1.1.1'
PORT = 5007

# Receiver function
def receive():
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM, socket.IPPROTO_UDP)
    sock.bind(('', PORT))

    mreq = struct.pack("4sl", socket.inet_aton(MULTICAST_GROUP), socket.INADDR_ANY)
    sock.setsockopt(socket.IPPROTO_IP, socket.IP_ADD_MEMBERSHIP, mreq)

    print("Receiver ready...\n")
    while True:
        data, addr = sock.recvfrom(1024)
        print(f"Received from {addr}: {data.decode()}")
```

```
# Sender function
def send():
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM, socket.IPPROTO_UDP)
    ttl = struct.pack('b', 1)
    sock.setsockopt(socket.IPPROTO_IP, socket.IP_MULTICAST_TTL, ttl)

    while True:
        msg = input("Enter message: ")
        sock.sendto(msg.encode(), (MULTICAST_GROUP, PORT))

# Run both sender & receiver
threading.Thread(target=receive, daemon=True).start()
send()
```

Output :

```
Output                                                    Clear
Enter message: Receiver ready...

hello group
Received from ('10.236.15.45', 60346): hello group
Enter message: ola have a nice day
Enter message: Received from ('10.236.15.45', 60346): ola have a nice day
```

Conclusion: The experiment successfully demonstrated group communication using multicast, where a single message was efficiently delivered to multiple processes. It showed how UDP multicast enables scalable and low-overhead communication in distributed systems, making it suitable for applications like group chat and real-time data sharing.

Class:	BE	**Semester:**	VIII
Course Code:	CSL801	**Course Name:**	Distributed Computing Lab

Name of Student:	Tanishka Jayesh Das
Roll No. :	34
Division:	1
Experiment No.:	4
Title of Experiment:	Implement Vector Clock Synchronization algorithms
Date of Submission:	29/1/2026
Date of Correction:	5/2/2026

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	4-5	2-3	1
Understanding	4-5	2-3	1
Journal work and timely submission	8-10	5-8	1-4

Checked by

Name of Faculty : **Dr. Swati Varma**

Signature :

Date :

EXPERIMENT 4

Aim: To Implement Vector Clock Synchronization algorithms

Objective:- To Implement Vector Clock Synchronization algorithms

Theory:

Vector clocks are an advanced logical clock mechanism used in distributed systems to capture causality more precisely than Lamport clocks.

In vector clocks:

- Each process maintains a vector (array) of size N (where N = number of processes).
- Each element represents the logical clock of a process.

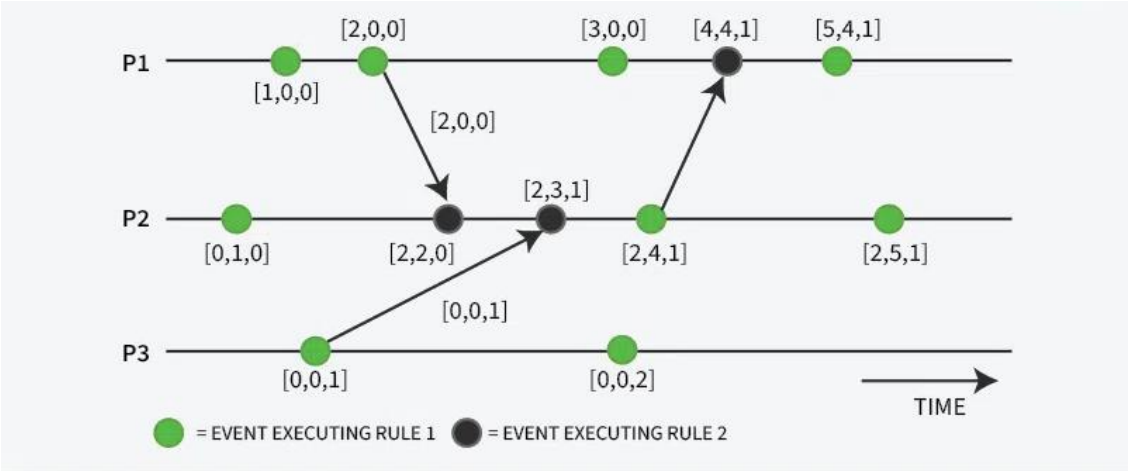
Rules of Vector Clock:

1. **Initialization:**
   - All vector values are initialized to 0.
   - Example for 3 processes:
     `P0 = [0,0,0], P1 = [0,0,0], P2 = [0,0,0]`
2. **Local Event:**
   - Increment its own index.
   - Example:
     If `P1` executes event → `[0,1,0]`
3. **Send Event:**
   - Increment its own clock.
   - Send entire vector with the message.
4. **Receive Event:**
   - Take **element-wise maximum** of:
     - Current vector
     - Received vector
   - Then increment its own index.

Key Advantage:

Vector clocks can:

- Detect **causal relationships**
- Identify **concurrent events** (which Lamport clocks cannot do precisely)



Code and output:

```
# Number of processes
n = 3

# Initialize vector clocks
P0 = [0]*n
P1 = [0]*n
P2 = [0]*n

# Function to print clocks
def show():
    print("P0:", P0)
    print("P1:", P1)
    print("P2:", P2)
    print()

# Event 1: P0 local event
P0[0] += 1
print("P0 performs local event")
show()

# Event 2: P0 sends message to P1
P0[0] += 1
msg = P0.copy()
print("P0 sends message to P1")
show()

# Event 3: P1 receives message
P1 = [max(P1[i], msg[i]) for i in range(n)]
P1[1] += 1
print("P1 receives message from P0")
show()

# Event 4: P2 local event
P2[2] += 1
print("P2 performs local event")
show()
```

OUTPUT:

```
Output                                                    Clear
P0 performs local event
P0: [1, 0, 0]
P1: [0, 0, 0]
P2: [0, 0, 0]

P0 sends message to P1
P0: [2, 0, 0]
P1: [0, 0, 0]
P2: [0, 0, 0]

P1 receives message from P0
P0: [2, 0, 0]
P1: [2, 1, 0]
P2: [0, 0, 0]

P2 performs local event
P0: [2, 0, 0]
P1: [2, 1, 0]
P2: [0, 0, 1]


=== Code Execution Successful ===
```

Conclusion: The experiment demonstrated vector clock synchronization, where each process maintains a vector to track events. It showed how vector clocks accurately capture causal relationships and help identify concurrent events, making them more precise than simple logical clocks in distributed systems.

Class:	BE	**Semester:**	VIII
Course Code:	CSL801	**Course Name:**	Distributed Computing Lab

Name of Student:	Tanishka Jayesh Das
Roll No. :	34
Division:	1
Experiment No.:	5
Title of Experiment:	Implement Election Algorithm
Date of Submission:	5/2/2026
Date of Correction:	12/2/2026

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	4-5	2-3	1
Understanding	4-5	2-3	1
Journal work and timely submission	8-10	5-8	1-4

Checked by

Name of Faculty : **Dr. Swati Varma**

Signature :

Date :

EXPERIMENT 5

AIM: To Implement Election Algorithm

Objective: Develop a program to implement Implement Election Algorithm

Theory:

Election Algorithms:

- The coordinator election problem is to choose a process from among a group of processes on different processors in a distributed system to act as the central coordinator.
- An election algorithm is an algorithm for solving the coordinator election problem. By the nature of the coordinator election problem, any election algorithm must be a distributed algorithm.

(a) Bully Algorithm

Background: any process Pi sends a message to the current coordinator; if no response in T time units, Pi tries to elect itself as leader. Details follow:

Algorithm for process Pi that detected the lack of coordinator

1. Process Pi sends an "Election" message to every process with higher priority.
2. If no other process responds, process Pi starts the coordinator code running and sends a message to all processes with lower priorities saying "Elected Pi"
3. Else, Pi waits for T' time units to hear from the new coordinator, and if there is no response à start from step (1) again.

Algorithm for other processes (also called Pi)

If Pi is not the coordinator then Pi may receive either of these messages from Pj

if Pi sends "Elected Pj"; [this message is only received if i < j]

Pi updates its records to say that Pj is the coordinator.

Else if Pj sends "election" message (i > j)

Pi sends a response to Pj saying it is alive

Pi starts an election.

(b) Election In A Ring => Ring Algorithm.

-assume that processes form a ring: each process only sends messages to the next process in the ring

- Active list: its info on all other active processes
- assumption: message continues around the ring even if a process along the way has crashed.

Background: any process Pi sends a message to the current coordinator; if no response in T time units, Pi initiates an election

1. initialize active list to empty.
2. Send an "Elect(i)" message to the right. + add i to active list.

If a process receives an "Elect(j)" message

(a) this is the first message sent or seen

initialize its active list to [i,j]; send “Elect(i)” + send “Elect(j)”

(b) if i != j, add i to active list + forward “Elect(j)” message to active list

(c) otherwise (i = j), so process i has complete set of active processes in its active list.

=> choose highest process ID + send “Elected (x)” message to neighbor

If a process receives “Elected(x)” message,

set coordinator to x

Example:

Suppose that we have four processes arranged in a ring: P1 à P2 à P3 à P4 à P1 …
P4 is coordinator
Suppose P1 + P4 crash
Suppose P2 detects that coordinator P4 is not responding
P2 sets active list to [ ]
P2 sends “Elect(2)” message to P3; P2 sets active list to [2]
P3 receives “Elect(2)”
This message is the first message seen, so P3 sets its active list to [2,3]
P3 sends “Elect(3)” towards P4 and then sends “Elect(2)” towards P4
The messages pass P4 + P1 and then reach P2
P2 adds 3 to active list [2,3]
P2 forwards “Elect(3)” to P3
P2 receives the “Elect(2) message

P2 chooses P3 as the highest process in its list [2, 3] and sends an “Elected(P3)” message

P3 receives the “Elect(3)” message

P3 chooses P3 as the highest process in its list [2, 3] + sends an “Elected(P3)” message

Code and output:

1. Bully Election Algorithm

```python
# Bully Election Algorithm Simulation

class BullyElection:
    def __init__(self, processes):
        self.processes = processes  # list of process IDs
        self.alive = {p: True for p in processes}
        self.coordinator = max(processes)

    def crash(self, pid):
        self.alive[pid] = False
        print(f"Process {pid} crashed.")

    def start_election(self, pid):
        print(f"\nProcess {pid} starts election")

        higher = [p for p in self.processes if p > pid and
self.alive[p]]

        if not higher:
```

```python
            self.coordinator = pid
            print(f"Process {pid} becomes new Coordinator")
        else:
            print(f"Process {pid} sends Election to {higher}")
            new_leader = max(higher)
            self.start_election(new_leader)

    def show_coordinator(self):
        print("Current Coordinator:", self.coordinator)


# Example
processes = [1, 2, 3, 4, 5]
bully = BullyElection(processes)

bully.show_coordinator()

bully.crash(5)         # coordinator crashes
bully.start_election(2)

bully.show_coordinator()
```

Output:

```
Current Coordinator: 5
Process 5 crashed.

Process 2 starts election
Process 2 sends Election to [3, 4]

Process 4 starts election
Process 4 becomes new Coordinator
Current Coordinator: 4
```

2. Ring Election Algorithm :

```python
# Ring Election Algorithm Simulation

class RingElection:
    def __init__(self, processes):
        self.processes = processes
        self.alive = {p: True for p in processes}
        self.coordinator = max(processes)

    def crash(self, pid):
        self.alive[pid] = False
        print(f"Process {pid} crashed.")
```

```python
    def start_election(self, starter):
        print(f"\nProcess {starter} starts election")

        active_list = []
        n = len(self.processes)
        index = self.processes.index(starter)

        for i in range(n):
            pid = self.processes[(index + i) % n]
            if self.alive[pid]:
                active_list.append(pid)

        print("Active Processes:", active_list)

        new_leader = max(active_list)
        self.coordinator = new_leader

        print(f"Process {new_leader} elected as Coordinator")

    def show_coordinator(self):
        print("Current Coordinator:", self.coordinator)


# Example
processes = [1, 2, 3, 4]
ring = RingElection(processes)

ring.show_coordinator()

ring.crash(4)
ring.start_election(2)

ring.show_coordinator()
```

Output:

```
Current Coordinator: 4
Process 4 crashed.

Process 2 starts election
Active Processes: [2, 3, 1]
Process 3 elected as Coordinator
Current Coordinator: 3
```

Conclusion:
The Bully and Ring election algorithms were successfully implemented to select a new coordinator in a distributed system when the current coordinator fails. In the Bully algorithm, the highest priority active process becomes the coordinator, whereas in the Ring algorithm, election messages circulate through all processes and the highest ID process is chosen. Both algorithms ensure reliable leader selection and maintain proper coordination among distributed processes.

Class:	BE	**Semester:**	VIII
Course Code:	CSL801	**Course Name:**	Distributed Computing Lab

Name of Student:	Tanishka Jayesh Das
Roll No. :	34
Division:	1
Experiment No.:	6
Title of Experiment:	Implement Mutual Exclusion Algorithm
Date of Submission:	12/2/2026
Date of Correction:	26/2/2026

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	4-5	2-3	1
Understanding	4-5	2-3	1
Journal work and timely submission	8-10	5-8	1-4

Checked by

Name of Faculty : **Dr. Swati Varma**

Signature :

Date :

EXPERIMENT 6

AIM: To implement mutual exclusion algorithm

Objective: Develop a program to implement mutual exclusion algorithm

THEORY:
Mutual exclusion : makes sure that concurrent process access shared resources or data in a serialized way. If a process , say Pi , is executing in its critical section, then no other processes can be executing in their critical sections
Token ring is one among mutual exclusion algorithm in distributed systems that does not involve with a server or coordinator.
Working
A token is passed from one node to another node in a logical order. A node received token has a proper right to enter the critical section. If a node being holding token does not require to enter critical section or access a shared resource, it just simply passes the token to the next node.
Advantages
- Process suffer from no starvation. Before entering a critical section, a process waits at most for the duration that all other processes enter and exit the critical section
- No hand shake messages are required to get the token. Then less overhead

Disadvantages
- If a token is lost for some reasons, another token must be regenerated
- Detecting lost token is difficult since there is no upper bound in the time a token takes to rotate the ring (no particular timeout)
- The ring must be reconstructed when a process crashes

Code and output:

1 **Token Based Algorithm**

🔷 **Singhal's Heuristic Algorithm**

📌 **Theory (Short Explanation for Viva)**

Singhal's algorithm is a **token-based distributed mutual exclusion algorithm**. Only the process holding the **token** can enter the critical section (CS).
- If a process wants to enter CS and does not have the token → it sends request messages.
- Token is passed based on request set.
- Ensures **no two processes enter CS at same time**.

CODE:
```
import threading
import time
import random

class Process:
    def __init__(self, pid, total_processes):
        self.pid = pid
        self.total = total_processes
        self.has_token = False
```

```
        self.request = False

    def request_cs(self):
        print(f"Process {self.pid} requesting Critical Section")
        self.request = True

    def enter_cs(self):
        print(f"Process {self.pid} ENTERED Critical Section")
        time.sleep(2)
        print(f"Process {self.pid} EXITING Critical Section")
        self.request = False

def singhal_algorithm(processes):
    token_holder = 0
    processes[token_holder].has_token = True

    while True:
        for p in processes:
            if p.request and processes[token_holder].has_token:
                processes[token_holder].has_token = False
                p.has_token = True
                token_holder = p.pid
                p.enter_cs()
                time.sleep(1)
        time.sleep(1)

# Main
n = 3
processes = [Process(i, n) for i in range(n)]

# Random request generator
def generate_requests():
    while True:
        time.sleep(random.randint(3, 6))
        p = random.choice(processes)
        p.request_cs()

threading.Thread(target=singhal_algorithm, args=(processes,), daemon=True).start()
threading.Thread(target=generate_requests, daemon=True).start()

time.sleep(20)
```

OUTPUT

```
Process 0 requesting Critical Section
Process 0 ENTERED Critical Section
Process 0 EXITING Critical Section
Process 2 requesting Critical Section
Process 2 ENTERED Critical Section
Process 2 EXITING Critical Section
Process 1 requesting Critical Section
Process 1 ENTERED Critical Section
Process 1 EXITING Critical Section
Process 0 requesting Critical Section
Process 0 ENTERED Critical Section
```

2️⃣ Non-Token Based Algorithm

🔷 Ricart & Agrawala Algorithm

📌 Theory (Short Explanation for Viva)

Ricart & Agrawala is a **permission-based distributed mutual exclusion algorithm**.

- When a process wants to enter CS:
  - It sends a **REQUEST message** to all other processes.
  - It waits for **REPLY from all processes**.
- Based on **timestamp comparison**, priority is decided.
- No token is used.

CODE

```
import threading
import time
import random

class RA_Process:
    def __init__(self, pid, total):
        self.pid = pid
        self.total = total
        self.timestamp = 0
        self.reply_count = 0
        self.requesting = False
        self.deferred = []

    def request_cs(self):
        self.requesting = True
        self.timestamp = time.time()
        self.reply_count = 0
        print(f"Process {self.pid} requesting CS at time {self.timestamp}")

        for p in processes:
            if p.pid != self.pid:
                p.receive_request(self.pid, self.timestamp)

    def receive_request(self, pid, timestamp):
        if (not self.requesting) or (timestamp < self.timestamp) or \
          (timestamp == self.timestamp and pid < self.pid):
```

```
        processes[pid].receive_reply()
        else:
            self.deferred.append(pid)

    def receive_reply(self):
        self.reply_count += 1
        if self.reply_count == self.total - 1:
            self.enter_cs()

    def enter_cs(self):
        print(f"Process {self.pid} ENTERED Critical Section")
        time.sleep(2)
        print(f"Process {self.pid} EXITING Critical Section")
        self.requesting = False

        for pid in self.deferred:
            processes[pid].receive_reply()


        self.deferred = []

# Main
n = 3
processes = [RA_Process(i, n) for i in range(n)]

def generate_requests():
    while True:
        time.sleep(random.randint(3, 6))
        p = random.choice(processes)
        p.request_cs()

threading.Thread(target=generate_requests, daemon=True).start()
time.sleep(20)
```

OUTPUT

```
Process 0 requesting CS at time 1771404465.2209086
Process 0 ENTERED Critical Section
Process 0 EXITING Critical Section
Process 1 requesting CS at time 1771404468.9175653
Process 1 ENTERED Critical Section
Process 1 EXITING Critical Section
Process 2 requesting CS at time 1771404473.2216926
Process 2 ENTERED Critical Section
Process 2 EXITING Critical Section
Process 1 requesting CS at time 1771404475.9189353
Process 1 ENTERED Critical Section
Process 1 EXITING Critical Section
Process 0 requesting CS at time 1771404479.2223616
Process 0 ENTERED Critical Section
Process 0 EXITING Critical Section
```

Conclusion:

- In this experiment, we successfully implemented **Token-Based (Singhal's Heuristic Algorithm)** and **Non-Token Based (Ricart & Agrawala Algorithm)** mutual exclusion algorithms in Python.
- The token-based approach ensures mutual exclusion by allowing only the process holding the token to enter the critical section. Since only one token exists in the system, it guarantees that no two processes can execute the critical section simultaneously.
- The non-token-based approach ensures mutual exclusion using request and reply messages along with timestamp ordering. A process enters the critical section only after receiving permission from all other processes, thereby preventing concurrent access.
- Both algorithms effectively maintain mutual exclusion, prevent race conditions, and ensure orderly access to shared resources in distributed systems. Thus, the objective of implementing and understanding mutual exclusion mechanisms was successfully achieved.

Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

Academic Year: 2025-26

Class:	BE	**Semester:**	VIII
Course Code:	CSL801	**Course Name:**	Distributed Computing Lab

Name of Student:	Tanishka Jayesh Das
Roll No. :	34
Division:	1
Experiment No.:	7
Title of Experiment:	Implement Deadlock Management in Distributed System
Date of Submission:	26/2/2026
Date of Correction:	12/3/2026

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	4-5	2-3	1
Understanding	4-5	2-3	1
Journal work and timely submission	8-10	5-8	1-4

Checked by

Name of Faculty : **Dr. Swati Varma**

Signature :

Date :

EXPERIMENT 7

AIM: To implement deadlock management in Distributed System

Objective: Develop a program to implement deadlock management in distributed system

THEORY:
The algorithm allows a process to request for multiple resources at a time. Special messages or probes circulated along edges of WFG. Deadlock exists if the probe is received back by the initiator.
Working:

1. When a process that requests for a resource, fails to get it and times out,
2. It generates a special probe message and sends it to the process (or processes) holding the requested resource.
    - The probe message (acts as a unique id) contains the following fields:
    - The id of the process just blocked
    - The id of the process sending this message
    - The id of the process to whom this message is being sent
3. On receipt of the probe message, the receiver checks to see if it itself is waiting for any resource.
4. If not, this means that the recipient is using the resource requested by the process that sent the probe message to it.
5. In this case, the recipient simply ignores the probe message.
6. On the other hand, if the recipient is itself waiting for any resource, it passes the probe message to the resource holding process (or processes)
7. But, before the probe is forwarded, the recipient does following modifications
8. The first field is left unchanged
9. The recipient changes the second field to its own process id.
10. The third field is changed to the id of the process that will be the new recipient of the probe.
11. Every new recipient of the probe message repeats this procedure.
12. If the probe message returns back to the original sender,(the process whose id is the first field of the message), a cycle exists and, the system is deadlocked.

Code and output:

CODE:
```
# Deadlock Detection using DFS (Wait-For Graph)

n = int(input("Enter number of processes: "))

# Initialize graph
graph = []
print("Enter Wait-For Graph matrix:")
```

```
for i in range(n):
    row = list(map(int, input().split()))
    graph.append(row)


visited = [0] * n
stack = [0] * n

def dfs(v):
    visited[v] = 1
    stack[v] = 1

    for i in range(n):
        if graph[v][i]:
            if not visited[i] and dfs(i):
                return True
            elif stack[i]:
                return True

    stack[v] = 0
    return False

def detect_deadlock():
     for i in range(n):
        visited[i] = 0
        stack[i] = 0

    for i in range(n):
        if not visited[i]:
            if dfs(i):
                return True

    return False

if detect_deadlock():
    print("Deadlock Detected in the system")
else:
    print("No Deadlock in the system")
```

OUTPUT:

```
Enter number of processes: 3
Enter Wait-For Graph matrix:
0 1 0
0 0 1
0 0 0
No Deadlock in the system
```

Conclusion:
The deadlock management algorithm detects deadlocks in a distributed system by identifying cycles in the Wait-For Graph (WFG). Using techniques like DFS or probe-based messaging, the system traces dependencies between processes, and if a cycle is found or a probe message returns to the initiating process, it confirms the presence of a deadlock. This method works efficiently in a distributed environment without requiring centralized control, ensuring accurate detection and better utilization of system resources.

Class:	BE	**Semester:**	VIII
Course Code:	CSL801	**Course Name:**	Distributed Computing Lab

Name of Student:	Tanishka Jayesh Das
Roll No. :	34
Division:	1
Experiment No.:	8
Title of Experiment:	Implement Task Assignment Approach
Date of Submission:	12/3/2026
Date of Correction:	6/4/2026

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	4-5	2-3	1
Understanding	4-5	2-3	1
Journal work and timely submission	8-10	5-8	1-4

Checked by

Name of Faculty : **Dr. Swati Varma**

Signature :

Date :

EXPERIMENT 8

Aim: To implement task assignment approach

Objective: Implement task assignment approach

Theory:

In task assignment approach, process is viewed as a collection of related tasks and these tasks are scheduled to suitable nodes so as to improve performance.

A process is split into pieces called tasks. This split occurs along natural boundaries, so that data transfers among the tasks is minimized. The amount of computation required by each task and the speed of each processor are known. The cost of processing each task on every node of the system is also known. The inter-process communication (IPC) costs between every pair of tasks are known.
Other constraints, such as resource requirements of the tasks and the available resources at each node, precedence relationships among the tasks, and so on, are also known.

Consider the following example:

Intertask communication costs						
	t_1	t_2	t_3	t_4	t_5	t_6
t_1	0	6	4	0	0	12
t_2	6	0	8	12	3	0
t_3	4	8	0	0	11	0
t_4	0	12	0	0	5	0
t_5	0	3	11	5	0	0
t_6	12	0	0	0	0	0

(a)

Execution costs		
Tasks	Nodes	
	n_1	n_2
t_1	5	10
t_2	2	∞
t_3	4	4
t_4	6	3
t_5	5	2
t_6	∞	4

(b)

The serial assignment cost can be calculated as follows:

$$\text{Serial assignment execution cost } (x) = x_{11} + x_{21} + x_{31} + x_{42} + x_{52} + x_{62}$$
$$= 5 + 2 + 4 + 3 + 2 + 4 = 20$$

$$\text{Serial assignment communication cost } (c) = c_{14} + c_{15} + c_{16} + c_{24} + c_{25} + c_{26} + c_{34} + c_{35} + c_{36}$$
$$= 0 + 0 + 12 + 12 + 3 + 0 + 0 + 11 + 0 = 38$$

$$\text{Serial assignment total cost} = x + c = 20 + 38 = 58$$

And the optimal assignment cost can be calculated as follows:

$$\text{Optimal assignment execution cost } (x) = x_{11} + x_{21} + x_{31} + x_{41} + x_{51} + x_{62}$$
$$= 5 + 2 + 4 + 6 + 5 + 4 = 26$$

$$\text{Optimal assignment communication cost } (c) = c_{16} + c_{26} + c_{36} + c_{46} + c_{56}$$
$$= 12 + 0 + 0 + 0 + 0 = 12$$

$$\text{Optimal assignment total cost} = x + c = 26 + 12 = 38$$

For implementing task assignment approach, take from the user all the parameters and then calculate the cost.

Code and output:

CODE:

```
# Task Assignment Approach

# Input
tasks = int(input("Enter number of tasks: "))
processors = int(input("Enter number of processors: "))

cost = []

print("Enter cost matrix:")

for i in range(tasks):
    row = list(map(int, input().split()))
    cost.append(row)

print("\nTask Assignment:")

total_cost = 0

# Assign each task to processor with minimum cost
for i in range(tasks):
    min_cost = cost[i][0]
    processor = 0

    for j in range(1, processors):
        if cost[i][j] < min_cost:
            min_cost = cost[i][j]
            processor = j
```

```
    total_cost += min_cost
    print(f"Task {i+1} assigned to Processor {processor+1} with cost {min_cost}")

print(f"\nTotal Assignment Cost: {total_cost}")
```

OUTPUT:

```
Enter number of tasks: 3
Enter number of processors: 3
Enter cost matrix:
10 20 30
5 15 25
8 18 28

Task Assignment:
Task 1 assigned to Processor 1 with cost 10
Task 2 assigned to Processor 1 with cost 5
Task 3 assigned to Processor 1 with cost 8

Total Assignment Cost: 23
```

Conclusion:
The task assignment approach assigns each task to the processor with the minimum execution cost, thereby reducing the overall execution time. The total assignment cost is minimized by selecting optimal processors for each task. This improves system performance and ensures efficient utilization of resources.

Class:	BE	**Semester:**	VIII
Course Code:	CSL801	**Course Name:**	Distributed Computing Lab

Name of Student:	Tanishka Jayesh Das
Roll No.:	34
Division:	1
Experiment No.:	9
Title of Experiment:	Implement Distributed shared Memory
Date of Submission:	6/4/2026
Date of Correction:	10/4/2026

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	4-5	2-3	1
Understanding	4-5	2-3	1
Journal work and timely submission	8-10	5-8	1-4

Checked by

Name of Faculty : **Dr. Swati Varma**

Signature :

Date :

EXPERIMENT 9

Aim: To implement distributed shared memory

Objective: Implement distributed shared memory using Semaphores

Theory:
A semaphore lock on the shared memory buffer reference allows processes accessing the shared memory to prevent a daemon from writing to the memory segment currently being accessed.

Semaphores are a synchronization mechanism used to coordinate the activities of multiple processes in a computer system. They are used to enforce mutual exclusion, avoid race conditions and implement synchronization between processes.

Semaphores provide two operations: wait (P) and signal (V). The wait operation decrements the value of the semaphore, and the signal operation increments the value of the semaphore.

The algorithm is as follows:

```
semaphore s = 1;

        Process 1 :                          Process 2 :
while (True)                         while (True)
{   nc1: /* non critical              {   nc2: /* non critical
            section */ ;                          section */
         wait(s);                             wait(s);
  crit1: /* critical section */ ;       crit2: /* critical section */
         signal(s);                           signal(s)
}                                    }
```

Code and output:

CODE:

```
# Distributed Shared Memory using multiprocessing and Semaphore

from multiprocessing import Process, Value, Semaphore

def child_process(shared_memory, sem):
    sem.acquire()  # Wait (P)
    print("Child Process accessing shared memory")

    shared_memory.value += 1
    print("Child updated value:", shared_memory.value)

    sem.release()  # Signal (V)

def parent_process(shared_memory, sem):
    sem.acquire()  # Wait (P)
    print("Parent Process accessing shared memory")
```

```
    shared_memory.value += 1
    print("Parent updated value:", shared_memory.value)


    sem.release()   # Signal (V)

if __name___== "__main__":
    # Shared memory (integer)
    shared_memory = Value('i', 0)

    # Semaphore
    sem = Semaphore(1)

    # Create child process
    p = Process(target=child_process, args=(shared_memory, sem))
    p.start()
    p.join()

    # Parent process
    parent_process(shared_memory, sem)
```

OUTPUT:

```
Child Process accessing shared memory
Child updated value: 1
Parent Process accessing shared memory
Parent updated value: 2
```

Conclusion:
Semaphores ensure mutual exclusion by allowing only one process to access the shared memory at a time. The wait (P) operation locks the shared resource, while the signal (V) operation releases it. This prevents race conditions and ensures correct and synchronized updates to shared memory.

Class:	BE	**Semester:**	VIII
Course Code:	CSL801	**Course Name:**	Distributed Computing Lab

Name of Student:	Tanishka Jayesh Das
Roll No. :	34
Division:	1
Experiment No.:	10
Title of Experiment:	Demonstrate Distributed File System
Date of Submission:	10/4/2026
Date of Correction:	10/4/2026

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	4-5	2-3	1
Understanding	4-5	2-3	1
Journal work and timely submission	8-10	5-8	1-4

Checked by

Name of Faculty : **Dr. Swati Varma**

Signature :

Date :

EXPERIMENT 10

Aim: To demonstrate Distributed File System

Objective:- Demonstrate Distributed File System using HDFS

Theory:-

HDFS employs a NameNode and DataNode architecture to implement a distributed file system that provides high-performance access to data across highly scalable Hadoop clusters.

File management tasks in hadoop

In order to perform operations on Hadoop like copy, delete, move etc., following steps can be used:

Basic operations:

1. Create a directory in HDFS at given path(s).

Usage:

hadoop fs -mkdir <paths>

2. List the contents of a directory.

Usage :

hadoop fs -ls <args>

3. See contents of a file

Same as unix cat command:

Usage:

hadoop fs -cat <path[filename]>

4. Copy a file from source to destination

This command allows multiple sources as well in which case the destination must be a directory.

Usage:

hadoop fs -cp <source> <dest>

5. Copy a file from/To Local file system to HDFS

copyFromLocal

Usage:

hadoop fs -copyFromLocal <localsrc> URI

Similar to put command, except that the source is restricted to a local file reference.

copyToLocal

Usage:

hadoop fs -copyToLocal [-ignorecrc] [-crc] URI <localdst>

Similar to get command, except that the destination is restricted to a local file reference.

6. Move file from source to destination.

Note:- Moving files across filesystem is not permitted.

Usage :

hadoop fs -mv <src> <dest>

7. Remove a file or directory in HDFS.

Remove files specified as argument. Deletes directory only when it is empty

Usage :

hadoop fs -rm <arg>

Steps for copying file

1) Go to Hadoop folder and then to sbin

C:\>cd C:\hadoop-2.8.0\sbin

2) Start namenode and datanode with this command, Two more cmd windows will open

C:\hadoop-2.8.0\sbin>start-dfs.cmd

3) Now start yarn through following command, Two more windows will open, one for yarn resource manager and one for yarn node manager

C:\hadoop-2.8.0\sbin>start-yarn.cmd

4) Create a directory named 'sample' in the hadoop directory using the following command

C:\hadoop-2.8.0\sbin> hdfs dfs -mkdir /sample

5) To verify if the directory is created

C:\hadoop-2.8.0\sbin>hdfs dfs -ls /

6) Copy text file from D drive to sample

C:\hadoop-2.8.0\sbin>hdfs dfs -copyFromLocal d:\rally.txt /sample

7) To verify if the file is copied

C:\hadoop-2.8.0\sbin>hdfs dfs -ls /sample

Code and output:

CODE:

```
# Distributed File System Simulation

def write_file():
    with open("shared_file.txt", "a") as fp:
        data = input("Enter data to write: ")
```

```
        fp.write(data + "\n")
    print("Data written successfully")

def read_file():
    try:
        with open("shared_file.txt", "r") as fp:
            print("\nFile contents:\n")


            for line in fp:
                print(line, end="")
    except FileNotFoundError:
        print("File does not exist yet.")

while True:
    print("\nDistributed File System Simulation")
    print("1. Write to shared file")
    print("2. Read from shared file")
    print("3. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        write_file()
    elif choice == 2:
        read_file()
    elif choice == 3:
        print("Exiting...")
        break
    else:
        print("Invalid choice")
```

OUTPUT:

```
Distributed File System Simulation
1. Write to shared file
2. Read from shared file
3. Exit
Enter your choice: 1
Enter data to write: Hello DFS
Data written successfully

Distributed File System Simulation
1. Write to shared file
2. Read from shared file
3. Exit
Enter your choice: 2

File contents:

Hello DFS

Distributed File System Simulation
1. Write to shared file
2. Read from shared file
3. Exit
Enter your choice: 3
Exiting...
```

Conclusion:
HDFS provides a scalable and fault-tolerant distributed storage system suitable for handling large datasets across multiple nodes. Compared to traditional file systems, HDFS is optimized for high-throughput data access and is widely used in big data applications like data analytics, log processing, and machine learning workloads.